Лабораторная работа №3: Построение полиморфного процессора событий (Event Dispatcher) для транзакций валидатора

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 3 из 10
Формат: Лабораторная работа, 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: Спроектировать событийно-ориентированную подсистему валидации проезда в городском транспорте с применением **полиморфизма**, абстрактных контрактов (typing.Protocol / abc.ABC) и архитектурного паттерна **Event Dispatcher**.
В реальном валидаторе Avtobys фиксация факта оплаты порождает цепочку полиморфных действий: - Запись финансового события в энергонезависимый локальный аудит-лог (SQLite / Append-only file). - Отправка push-уведомления пассажиру в мобильное приложение (если оплата была через Kaspi QR / Onay / Avtobys). - Проверка на фрод (повторное прикладывание одной и той же карты в течение 2 минут на одном валидаторе). - Обновление агрегированных счетчиков рейса для диспетчерской службы.
Ваша задача — исключить монолитные условные операторы и создать расширяемую шину событий, в которую новые обработчики подключаются полиморфно без правки существующего кода.

---

2. Структура репозитория и требования к сдаче

Работа выполняется в папке lab03_polymorphic_events/:

```
lab03_polymorphic_events/
├── dispatcher/
│   ├── __init__.py
│   ├── events.py       # Иерархия неизменяемых событий (dataclasses)
│   ├── contracts.py    # Протоколы обработчиков (Handler Protocols)
│   ├── bus.py          # Реализация EventDispatcher
│   └── handlers.py     # Обработчики: Audit, Notification, Fraud, Metrics
├── tests/
│   ├── __init__.py
│   └── test_dispatcher.py # Тесты диспетчера, изоляции ошибок и подписок
└── README.md           # Архитектурное описание полиморфного механизма
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Моделирование типизированных событий	4

Блок	Задание	Баллы
	(events.py) с иммутабельностью	
Блок 2	Разработка EventDispatcher с поддержкой полиморфной диспетчеризации	8
Блок 3	Реализация 3+ независимых обработчиков и изоляция исключений	5
Блок 4	Написание юнит-тестов с мок-объектами (unittest.mock / pytest)	3

---

3. Практические задания

Блок 1. Моделирование событий предметной области (4 балла)

В dispatcher/events.py создайте базовое событие и его специализации, используя @dataclass(frozen=True) для обеспечения неизменяемости:

1. BaseTransactionEvent: базовый класс с полями event_id: str, timestamp: datetime, bus_id: str, amount: Decimal.
2. CardTapEvent: дополняется card_id: str, remaining_balance: Decimal.
3. QrPaymentEvent: дополняется qr_token: str, payment_provider: str (например, "Kaspi", "Halyk").
4. BluetoothBeepEvent: дополняется beacon_id: str, signal_strength_rssi: int.
5. SecurityAlertEvent: событие подозрительной активности (card_id: str, alert_reason: str).

Блок 2. Проектирование EventDispatcher и контрактов (8 баллов)

В dispatcher/contracts.py определите протокол обработчика:

```
from typing import Protocol, runtime_checkable
from dispatcher.events import BaseTransactionEvent

@runtime_checkable
class EventHandler(Protocol):
    def handle(self, event: BaseTransactionEvent) -> None:
        """Полиморфная обработка события."""
        ...
```

В dispatcher/bus.py реализуйте диспетчер: - Метод subscribe(event_type: type[BaseTransactionEvent], handler: EventHandler) -> None — регистрация обработчика. - Метод unsubscribe(event_type: type[BaseTransactionEvent], handler: EventHandler) -> None — отписка. - Метод dispatch(event: BaseTransactionEvent) -> int — полиморфная доставка события всем зарегистрированным подписчикам данного типа события. Возвращает количество успешно вызванных обработчиков. - **Изоляция ошибок**: если один из обработчиков падает с ошибкой, диспетчер обязан перехватить её, залогировать и продолжить вызов остальных обработчиков (сбой отправки SMS не должен блокировать финансовый аудит!).

Блок 3. Реализация полиморфных обработчиков (5 баллов)

В dispatcher/handlers.py реализуйте: 1. AuditLogHandler: записывает строковое представление каждого события в список audit_records. 2. FraudDetectionHandler: отслеживает интервалы между прикладываниями одной и той же карты (CardTapEvent). Если интервал меньше 120 секунд в одном автобусе — генерирует событие SecurityAlertEvent («Double Tap Fraud»). 3. PassengerNotifierHandler: симулирует отправку push/SMS уведомления для пассажиров при событиях оплаты.

Блок 4. Автоматическое тестирование (pytest) (3 балла)

В tests/test_dispatcher.py напишите тесты: 1. Тест полиморфной доставки: отправка QrPaymentEvent вызывает только обработчики QR и общие транзакционные обработчики. 2. Тест изоляции исключений: падающий с RuntimeError мок-обработчик не прерывает работу других обработчиков в цепочке. 3. Тест детектора повторных прикладываний: два события с одной картой с разницей в 30 секунд генерируют алерт.
pytest tests/test_dispatcher.py -v

---

4. Вопросы для защиты лабораторной работы

1. Чем отличается вызов полиморфного обработчика от жесткой конструкции if isinstance(event, ...):?
2. Почему события в распределенных системах проектируются неизменяемыми (frozen=True)?
3. В каких случаях предпочтительнее использовать typing.Protocol вместо явного наследования от базового класса обработчика?
4. Как реализовать приоритет вызова обработчиков, если аудит должен выполняться гарантированно раньше нотификации?